🎓 Detailed & Precise Viva Q&A — Unified ML Portal

> **How to use:** Each answer is self-contained — read the question, think of your answer, then verify. Pay attention to the **"Precise Point"** callouts; examiners love those.

🔷 SECTION 1: Machine Learning Fundamentals

Q1. What is Machine Learning? How is it different from traditional programming?

> **Traditional Programming:**
> You write explicit rules → Computer follows them.
> `if credit_score > 700 and income > 50000: approve`
>
> **Machine Learning:**
> You give data + correct answers → Algorithm learns the rules itself.
> The model finds patterns like: "*Low credit + high income → 72% approval*" — without you coding that rule.
>
> **Key difference:** In ML, the rules (weights/parameters) are learned from data, not written by hand.

Q2. What are the 3 types of Machine Learning? Give examples from your project.

Type	How it works	Real project example
Supervised	Labeled data (input → correct answer)	Loan approval (we know past approvals/rejections)
Unsupervised	No labels; finds hidden structure	Customer segmentation (K-Means) — not in our project
Reinforcement	Agent learns from rewards/penalties	Game-playing AI like AlphaGo — not in our project

> **Precise Point:** All 5 models in our project are **Supervised Learning** — every training example has a known label (churn/no churn, approved/rejected, mask/no mask, etc.)

Q3. What is a Feature and a Label? Give examples from each model.

- **Feature (X):** An input variable the model uses to make a prediction.
- **Label (Y):** The correct output we want to predict.

Model	Example Features	Label
Churn	Tenure, SatisfactionScore, NumberOfOrders	Churn = 0 or 1
Loan	CreditScore, Income, LoanAmount	Approved = 1, Rejected = 0
Face Mask	128×128 pixel values (RGB)	Mask / No Mask / Incorrect
Fruit/Veg	224×224 pixel values	36 class labels
Audio	Mel-Spectrogram pixels	Language (10 classes)

Q4. What is a training set and a test set? Why do we separate them?

- **Training set (~80%):** Data the model learns from. Parameters are adjusted based on this.
- **Test set (~20%):** Data the model has never seen. Used to measure real-world performance.

Why separate? If we tested on the same data we trained on, the model could simply memorise all answers and appear 100% accurate — but completely fail on new data. The test set simulates unseen real-world input.

Analogy: Studying textbook questions (training) vs. actual exam questions (testing). If the exam uses the same questions, it doesn't test genuine understanding.

Q5. What is Overfitting? What is Underfitting? How do you fix them?

```
            Training Error    Test Error     Problem
Underfitting:    HIGH              HIGH       Model too simple
Overfitting:     LOW               HIGH       Model too complex / memorised
Good Fit:        LOW               LOW        Learned the actual pattern
```

Causes of Overfitting: - Too many parameters relative to training data - Model trained too long (too many epochs) - Noisy data that the model learned as patterns

How to fix Overfitting: - More training data - Regularisation (L1/L2 penalties in XGBoost) - Dropout layers (randomly disable neurons during training) - Early Stopping (stop training when validation loss stops improving) - Cross-validation

How to fix Underfitting: - Use a more complex model - Add more features (feature engineering) - Train for more epochs (for neural networks) - Reduce regularisation

---

Q6. Explain the Bias-Variance Tradeoff.

- **Bias** = Error from wrong assumptions in the model. High bias → underfit.
  Example: Fitting a straight line to a curved pattern.

- **Variance** = Error from sensitivity to small fluctuations in training data. High variance → overfit.
  Example: A model that perfectly follows every training point, including noise.

The Tradeoff: Reducing bias increases variance and vice versa. We want **both** to be low — this is the "sweet spot."

In our project: Gradient Boosting and XGBoost balance this by combining many weak (high bias) trees — the ensemble reduces variance while keeping bias low.

Q7. What is Cross-Validation? Why is it better than a single train/test split?

k-Fold Cross-Validation: 1. Split data into k equal parts ("folds"), e.g., k=5 2. Train on 4 folds, validate on the remaining 1 3. Repeat 5 times, each time using a different fold as validation 4. Average the 5 accuracy results

Why better? A single split gives one accuracy number — could be lucky/unlucky depending on which data ended up where. Cross-validation uses ALL data for both training and validation (at different times), giving a more reliable accuracy estimate.

Precise Point: We used `random_state=42` to ensure the same random split every time, making experiments reproducible.

Q8. What is Data Leakage? Why is it dangerous?

Data Leakage = When information from the test set accidentally influences the training process.

Example of leakage: If you scale the entire dataset first (calculating mean/std from all data), then split into train/test — the scaler has already "seen" the test data's distribution. Your model appears to perform well in testing but will fail on truly new data.

Correct approach: Fit the scaler ONLY on training data. Then transform test data using those same training parameters. `python scaler.fit(X_train) # Learn mean/std from TRAIN only X_train_scaled = scaler.transform(X_train) X_test_scaled = scaler.transform(X_test) # Apply same params to TEST`

Pipeline saves you: `sklearn.pipeline.Pipeline` automatically applies this correctly.

🔷 SECTION 2: Algorithms — Deep Dive

Q9. What is a Decision Tree? How does it split data?

A Decision Tree is a flowchart of if-else questions. At each node, it chooses the best question (feature + threshold) that most cleanly separates the classes.

Splitting Criteria:

1. **Gini Impurity:** Measures the probability of misclassifying a random sample.
2. `Gini = 1 - Σ(pᵢ²)` where p_i = fraction of samples of class i
3. A pure node (all one class) has Gini = 0
4. **Entropy / Information Gain:**
5. `Entropy = -Σ(pᵢ × log₂(pᵢ))`
6. `Information Gain = Entropy(parent) - weighted_avg(Entropy(children))`
7. We pick the split with the highest Information Gain

Example: `Is CreditScore > 700? ├── Yes → Is Income > 50000? │ ├── Yes → APPROVED │ └── No → REJECTED └── No → REJECTED`

Weakness: A single deep tree easily overfits. That's why we use ensembles (Random Forest, Gradient Boosting).

Q10. What is Gradient Boosting? Explain step-by-step.

Gradient Boosting is an **ensemble method** — it builds many weak learners (shallow decision trees) **sequentially**, where each tree corrects the errors of the previous one.

Step-by-step algorithm: 1. Start with a naive prediction: predict the mean of y (for regression) or log-odds (for classification) 2. Calculate **residuals** = actual − predicted (the errors) 3. Train a small decision tree to predict those residuals 4. Update the model: new prediction = old prediction + learning_rate × tree_prediction 5. Repeat steps 2–4 for `n_estimators` iterations

Key parameters: - `n_estimators=50` → How many trees (more = slower, potentially better) - `learning_rate=0.1` → Shrinks each tree's contribution (smaller = slower but more accurate) - `max_depth=3` → Depth of each weak tree (shallow = weak learner, works best for GB)

Why it's called "Gradient" Boosting?
Because it minimises a loss function by using **gradient descent** — each new tree moves in the direction that reduces the loss the most.

Q11. What is XGBoost? How is it different from standard Gradient Boosting?

XGBoost (Extreme Gradient Boosting) is an optimised implementation of gradient boosting with several improvements:

Feature	Standard Gradient Boosting	XGBoost
Speed	Sequential, slow	Parallel tree construction
Regularisation	None built-in	L1 (Lasso) + L2 (Ridge)
Missing values	Requires imputation	Handles natively

Feature	Standard Gradient Boosting	XGBoost
Memory	Less efficient	Column block structure for cache efficiency
Cross-validation	External	Built-in `cv()` method
Pruning	Pre-pruning (max_depth)	Post-pruning (gamma parameter)

L1 vs L2 Regularisation in XGBoost: - L1 (alpha): Adds sum of absolute values of weights → drives some weights to exactly 0 (feature selection) - L2 (lambda): Adds sum of squared weights → shrinks all weights proportionally

---

Q12. What is a Convolutional Neural Network (CNN)? Why do we use it for images?

Why not a regular neural network for images?
A 224×224 RGB image has 224 × 224 × 3 = **150,528 input values**. If we connected these to 1000 neurons in a regular (Dense) layer, that's 150 million parameters — impossible to train on small datasets.

CNN key idea: Instead of connecting every pixel to every neuron, use **local filters** that slide over the image and detect local patterns.

Layers explained:

1. **Conv2D(filters=32, kernel_size=3×3):**
2. A 3×3 filter slides across the image
3. Calculates dot product at each position → creates a "feature map"

4. Early layers detect edges, later layers detect shapes/faces

5. **MaxPooling2D(pool_size=2×2):**

6. Takes the maximum value in each 2×2 region → shrinks feature maps by 75%

7. Reduces computation, adds spatial invariance (slight shifts don't matter)
8. **GlobalAveragePooling2D:**
9. Averages each entire feature map into one number
10. Converts 3D feature tensor → 1D feature vector
11. **Dense(256, activation='relu'):**
12. Fully connected layer — combines all features for final decision
13. **Dense(num_classes, activation='softmax'):**
14. Final output — probabilities for each class

Q13. What is ReLU? Why is it preferred over Sigmoid for hidden layers?

ReLU (Rectified Linear Unit): `f(x) = max(0, x)` Output: 0 if x < 0, else x (passes through positive values unchanged)

Why better than Sigmoid for hidden layers?

Problem with Sigmoid	How ReLU Fixes It
Vanishing gradients: output saturates near 0 and 1, gradient ≈ 0 → deep networks stop learning	ReLU gradient = 1 for all positive x → gradients flow freely
Computationally expensive (exponential)	Computationally trivial (just a max operation)
Outputs not zero-centered	Not zero-centered but still better in practice

Where Sigmoid IS used: Binary output layers (predict probability 0–1 for binary classification).

Q14. What is Softmax activation? How does it work?

Softmax converts a vector of raw scores (logits) into a probability distribution that sums to 1.

Formula: `softmax(xᵢ) = e^xᵢ / Σ(e^xⱼ) for all j in all classes`

Example: `Raw logits: [2.1, 0.3, 1.5] e^2.1 = 8.17, e^0.3 = 1.35, e^1.5 = 4.48 Sum = 14.00 Softmax: [8.17/14, 1.35/14, 4.48/14] = [0.584, 0.096, 0.320] Predicted class = 0 (highest probability: 58.4%)`

Why exponents? It amplifies differences — the highest score gets disproportionately more probability. Also ensures all outputs are positive.

Used in: The final Dense layer of all our multi-class models (Fruit/Veg: 36 classes, Audio: 10 classes).

Q15. What is Transfer Learning? Why did we use MobileNetV2?

Transfer Learning = Taking a model already trained on a large dataset and reusing its learned features for a new (smaller) task.

Analogy: A surgeon learning to use a new scalpel doesn't need to re-learn anatomy from scratch. Similarly, MobileNetV2 already "knows" how to detect edges, textures, shapes — we only teach it which of our categories match those shapes.

Our approach — 2 phases: 1. **Phase 1 (Frozen base):** Freeze all MobileNetV2 layers. Train ONLY our new classification head (Dense layers). Fast training, no risk of destroying ImageNet knowledge. 2. **Phase 2 (Fine-tuning):** Unfreeze the LAST FEW layers of MobileNetV2. Train with a very small learning rate. The model's feature detectors slightly adapt to our specific domain.

Why MobileNetV2 specifically? - Designed for efficiency — uses **Depthwise Separable Convolutions** which reduce FLOP count by ~8-9x vs standard convolutions - Pre-trained

on ImageNet (1.4 million images, 1000 classes) - Input: 224×224×3 (perfect for our fruit/veg and audio models) - Much less GPU needed than VGG16/ResNet50

Q16. What are Depthwise Separable Convolutions? (Why MobileNet is efficient)

Standard Convolution: One filter processes all input channels simultaneously. Cost: `D_K × D_K × M × N × D_F × D_F` operations (M input channels, N output channels, D_K kernel size, D_F feature map size)

Depthwise Separable Convolution: Split into two steps: 1. **Depthwise Conv:** One filter per input channel (processes channels independently) 2. **Pointwise Conv (1×1):** Combine results across channels

Result: ~8-9x fewer computations with similar accuracy. This is why MobileNet runs on phones.

🔷 SECTION 3: Preprocessing & Data Processing

Q17. What is StandardScaler? What does it do mathematically?

StandardScaler transforms each feature to have **mean = 0** and **standard deviation = 1** (Z-score normalisation).

Formula: `z = (x - μ) / σ` Where μ = mean of that feature in training data, σ = standard deviation

Example:
```
Income column: [30000, 50000, 70000, 100000] μ = 62500, σ = 29155
Scaled: [-1.12, -0.43, 0.26, 1.29] ← now all roughly between -2 and +2
```

Why needed? Gradient-based algorithms and distance-based algorithms (KNN, SVM, Neural Networks) are sensitive to feature scale. Without scaling, income (50,000)

dominates age (30) in distance calculations, even though both may be equally important.

Important: `scaler.fit()` on TRAIN only. `scaler.transform()` on both train and test.

Q18. What is the difference between LabelEncoder and OneHotEncoder?

	LabelEncoder	OneHotEncoder
Input	['Cat', 'Dog', 'Bird']	['Cat', 'Dog', 'Bird']
Output	[0, 1, 2]	[[1,0,0],[0,1,0],[0,0,1]]
Implies order?	✅ Yes (Dog > Cat numerically)	❌ No (all equidistant)
Use for	Ordinal data, or target labels	Nominal (unordered) categories
Problem	May mislead tree models into thinking Cat < Dog	Increases dimensionality

When to use which: - LabelEncoder on the **target (y)** column is always fine (the label is just an identifier) - LabelEncoder on **features** only if the category is naturally ordered (Low < Medium < High) - OneHotEncoder for nominal features like Gender, PreferredPaymentMode

---

Q19. What is a Mel-Spectrogram and how does audio classification work?

Step 1: Load audio ```python y, sr = librosa.load('audio.wav', sr=22050)

y = amplitude array, sr = 22050 samples/second

```

**Step 2: Trim silence** `python y, _ = librosa.effects.trim(y, top_db=20) # Remove quiet parts < 20dB`

**Step 3: Generate Mel-Spectrogram** `python S = librosa.feature.melspectrogram(y=y, sr=22050, n_mels=128) S_dB = librosa.power_to_db(S, ref=np.max)` - X-axis: time - Y-axis: frequency in **Mel scale** (logarithmic — mimics human perception) - Color/brightness: intensity in decibels

**Step 4: Save as image → Feed to CNN** The spectrogram image (224×224) is then treated exactly like a photo and classified by MobileNetV2.

**Why Mel scale?**
Human hearing perceives pitch logarithmically — the difference between 100 Hz and 200 Hz sounds the same as between 1000 Hz and 2000 Hz. Mel scale assigns equal perceptual spacing, making it ideal for speech/language analysis.

---

### Q20. What is Image Normalisation? Why do we divide pixels by 255?

Raw pixel values are integers from **0 to 255** (8-bit per channel).

Neural networks train better when inputs are in a **small range** (typically 0–1 or -1 to 1) because: 1. Gradient descent converges faster (gradients stay in a useful magnitude) 2. Activation functions like Sigmoid and Tanh work in these ranges 3. Avoids numerical instability with large numbers

`python img_array = np.array(image) / 255.0 # Scales [0, 255] → [0.0, 1.0]`

**MobileNetV2 specifically** expects inputs preprocessed with `tf.keras.applications.mobilenet_v2.preprocess_input()` which scales to **[-1, 1]**
```

range.

🔷 SECTION 4: Libraries — Precise Explanations

Q21. What is the difference between `.fit()`, `.transform()`, and `.fit_transform()`?

Method	What it does	When to use
`.fit(X)`	Learns parameters from X (e.g., mean, std for scaler; categories for encoder)	On training data only
`.transform(X)`	Applies already-learned parameters to X	On any data (train or test)
`.fit_transform(X)`	Fits then transforms in one step	ONLY on training data

Rule: Never call `.fit()` or `.fit_transform()` on test data. This would cause data leakage (the scaler would learn from test statistics and give unfair results).

```python

# CORRECT ✅

scaler.fit(X_train) X_train_scaled = scaler.transform(X_train) X_test_scaled = scaler.transform(X_test) # Uses training mean/std

# WRONG ❌ — leakage
```

X_test_scaled = scaler.fit_transform(X_test) # Learns from test data! ```

Q22. What is an sklearn Pipeline? Why is it so important?

A **Pipeline** chains multiple processing steps into one object. Calling `pipeline.fit(X_train)` applies each step in order, and `pipeline.predict(X)` processes new data through all steps automatically.

Our Loan model pipeline: `python pipeline = Pipeline([ ('preprocessor', ColumnTransformer([ ('num', StandardScaler(), numeric_cols), ('cat', OneHotEncoder(), categorical_cols) ])), ('model', XGBClassifier(n_estimators=100)) ]) pipeline.fit(X_train, y_train) pipeline.predict(X_new) # Automatically preprocesses then predicts`

Benefits: 1. **No data leakage** — Pipeline guarantees scaler only sees training data 2. **Single object to save** — `pickle.dump(pipeline)` saves ALL steps 3. **Clean code** — No separate preprocessing steps to manage 4. **Cross-validation safe** — `cross_val_score(pipeline, X, y)` correctly refits preprocessing inside each fold

---

Q23. What is ColumnTransformer? Why do we need it?

A **ColumnTransformer** applies different transformations to different columns of the dataframe — essential when you have both numeric and categorical data.

```python from sklearn.compose import ColumnTransformer

preprocessor = ColumnTransformer([ ('num', StandardScaler(), ['income', 'loan_amount', 'credit_score']), ('cat', OneHotEncoder(), ['gender', 'education', 'employment_status']) ]) ```

**Without ColumnTransformer:** You'd manually split columns, apply different transforms, then awkwardly rejoin them — error-prone and doesn't integrate with Pipeline cleanly.
```

Q24. What is Flask? Explain the request-response cycle in your project.

> **Flask** is a lightweight Python web framework (WSGI-based). It maps URL paths to Python functions.
>
> **Complete cycle for Loan Prediction:** `1. User opens browser → types localhost:5000/loan 2. Flask serves loan.html page (GET /loan) 3. JavaScript fetches GET /api/loan/features → {fields, options} 4. User fills form, clicks "Predict" 5. JavaScript sends POST /api/loan/predict → {CreditScore: 700, Income: 60000, ...} 6. Flask receives JSON via request.json 7. Converts to DataFrame → passes to pipeline.predict() 8. Returns jsonify({'prediction': 1, 'probability': 0.87, 'result': 'APPROVED'}) 9. JavaScript receives JSON → displays "✅ APPROVED — 87% confidence"`
>
> **Key Flask concepts:** - `@app.route('/path', methods=['GET','POST'])` — declares a URL route - `request.json` — parses incoming JSON body - `jsonify()` — serializes Python dict to JSON response - `render_template('page.html')` — serves HTML from the templates/ folder - `debug=True` — auto-reloads server on code change (dev only)

Q25. What is pickle? What problem does it solve?

> **pickle** is Python's built-in serialisation module. It **converts Python objects to binary bytes** and saves them to disk — then restores them identically later.
>
> **Why needed?** Training a model takes time (minutes to hours). Without saving, you'd retrain every time the server starts.
>
> ```python import pickle
>
> # After training:
>
> with open('model.pkl', 'wb') as f: # 'wb' = write binary pickle.dump(pipeline, f)

On server start:

with open('model.pkl', 'rb') as f: # 'rb' = read binary pipeline = pickle.load(f) ```

Pickle saves everything — the XGBClassifier's tree structure, the StandardScaler's learned mean/std, the OneHotEncoder's category mappings — all in one `.pkl` file.

Problem we faced: Using `'w'` instead of `'wb'` (text mode instead of binary) can corrupt pickle files → `UnpicklingError`.

Q26. What is librosa? What are its key functions used in the audio model?

librosa is a Python library for audio analysis.

Function	What it does
`librosa.load(path, sr=22050)`	Load audio → returns (amplitude array, sample_rate). Resamples to sr if needed.
`librosa.effects.trim(y, top_db=20)`	Removes silence below 20dB threshold from start/end
`librosa.feature.melspectrogram(y, sr, n_mels=128)`	Computes Mel-Spectrogram (2D array of power per mel band per time frame)
`librosa.power_to_db(S, ref=np.max)`	Converts power spectrogram to decibels (log scale, more perceptually meaningful)

`sr=22050` **explained:** 22,050 samples per second = standard for audio (half the CD quality of 44,100 Hz). The Nyquist theorem guarantees we can represent frequencies up to 11,025 Hz — more than enough for speech.

🔷 SECTION 5: Evaluation Metrics

Q27. Define Accuracy, Precision, Recall, and F1-Score. When does each matter?

Given a confusion matrix: `Predicted Positive Predicted Negative Actual Positive: TP FN Actual Negative: FP TN`

Metric	Formula	Real meaning
Accuracy	(TP+TN)/(TP+TN+FP+FN)	% predictions correct overall
Precision	TP/(TP+FP)	Of all I flagged as positive, how many actually were?
Recall	TP/(TP+FN)	Of all actual positives, how many did I catch?
F1-Score	2×(P×R)/(P+R)	Harmonic mean — useful when classes are imbalanced

When each matters: - **Medical diagnosis (cancer):** High Recall is critical — missing an actual case (FN) is catastrophic - **Email spam filter:** High Precision is critical — flagging real emails as spam (FP) annoys users - **Loan default prediction:** High Recall — missing an actual defaulter costs the bank money - **Imbalanced dataset:** Never use accuracy alone — a model that always says "no default" can be 95% accurate on a 5% default dataset, but is completely useless

Q28. What is AUC-ROC? What does it tell you?

ROC Curve plots True Positive Rate (Recall) vs False Positive Rate at every possible decision threshold.

AUC (Area Under Curve): - AUC = 1.0 → Perfect model - AUC = 0.5 → Random guessing (diagonal line) - AUC = 0.0 → Perfectly wrong

Why useful: Accuracy is threshold-dependent (usually 0.5). AUC measures performance across ALL thresholds — shows how well the model discriminates between classes regardless of threshold choice.

In our project: Loan model can output `predict_proba()` → probability of approval. AUC tells us how well it separates approvals from rejections.

🔷 SECTION 6: Project-Specific Questions

Q29. Why did you choose Gradient Boosting for churn and XGBoost for loans?

- **Churn model:** The dataset was relatively smaller and well-structured. Gradient Boosting was sufficient and is available directly in scikit-learn, making it easier to integrate with the rest of the sklearn pipeline.
- **Loan model:** Required a more robust pipeline with complex preprocessing (multiple column types). XGBoost specifically handles missing values natively (loan datasets often have missing income entries), and its regularisation makes it less prone to overfitting on financial data with multicollinearity.

Q30. What is lazy loading? Why did you implement it?

Lazy loading = Load a resource only when it's first needed, not at startup.

```python models_dict = {'loan': None, 'churn': None}

def get_loan_model(): if models_dict['loan'] is None: models_dict['loan'] = pickle.load(open('pipeline.pkl', 'rb')) return models_dict['loan'] ```
```

> **Why?** The Flask server can boot even if some model files are missing or have errors. If the face mask model hasn't been trained yet but the loan model works, users can still access the loan endpoint. Without lazy loading, one missing file would crash the entire server on startup.

Q31. What does `predict_proba()` return? How is it different from `predict()`?

> - `predict(X)` → Returns the predicted **class label** (e.g., `1` or `0`)
> - `predict_proba(X)` → Returns a **probability array** for each class
>
> ```python pipeline.predict([[700, 60000, 200000]]) # → [1] pipeline.predict_proba([[700, 60000, 200000]]) # → [[0.13, 0.87]]
>
> # Class 0 (Rejected): 13%, Class 1 (Approved): 87%
>
> ---
>
> ```
>
> We display this probability as the **confidence score** in our UI: "APPROVED — 87% confidence."

Q32. What is `random_state=42`? Does the number 42 matter?

> `random_state` is a seed for the pseudo-random number generator. Setting it ensures the same "random" splits, initializations, and shuffles happen every run — making experiments **reproducible**.
>
> **42 has no special mathematical significance** — it's a convention from "The Hitchhiker's Guide to the Galaxy" (where 42 is the "answer to everything"). Any fixed integer (0, 1, 100)

> would work equally well. The important thing is to use the same value consistently across your code.

Q33. What is `include_top=False` in MobileNetV2? What do you add instead?

> `python base = tf.keras.applications.MobileNetV2(weights='imagenet', include_top=False)`
>
> `include_top=False` removes the final classification layers of MobileNetV2 that were trained to output 1000 ImageNet classes. We don't want those — we want our own classes (36 for fruits/veg, 10 for languages).
>
> **We add our own classification head:** `python x = base.output x = GlobalAveragePooling2D()(x) x = Dense(256, activation='relu')(x) x = Dropout(0.5)(x) output = Dense(num_classes, activation='softmax')(x) model = Model(inputs=base.input, outputs=output)`

Q34. What is Dropout? How does it prevent overfitting?

> **Dropout(rate=0.5)** randomly sets 50% of neuron outputs to 0 during each training step.
>
> **Why this prevents overfitting:** - Neurons can't co-adapt (rely on each other) - Forces each neuron to independently learn useful features - Acts like training an ensemble of many different sub-networks
>
> **At test time:** Dropout is automatically turned off — all neurons active. The outputs are scaled by (1 - dropout_rate) to maintain consistent expected values.
>
> **Important:** Dropout is only active during `model.fit()`. During `model.predict()` or `model.evaluate()`, it's disabled automatically.

Q35. What is the difference between `.keras`, `.h5`, and `.pkl` model files?

Format	Library	Contents	When to use
`.pkl`	Python pickle	Any Python object — sklearn models, pipelines, scalers	Tabular ML models (our churn & loan models)
`.h5`	HDF5 (via Keras)	Neural network weights, architecture, optimizer state	TensorFlow/Keras models (older format)
`.keras`	TensorFlow	Same as .h5 but newer format — recommended in TF 2.12+	TensorFlow/Keras models (our CNN models)

Loading: ```python

sklearn/XGBoost

model = pickle.load(open('model.pkl', 'rb'))

Keras/TensorFlow

model = tf.keras.models.load_model('model.keras') ```

Q36. What does `base.trainable = False` do? When do you set it to True?

`python base.trainable = False # Freeze all layers`

Setting `trainable = False` prevents the MobileNetV2 pre-trained weights from being updated during training. This effectively locks in the ImageNet-learned features.

Phase 1 (Warm-up): `base.trainable = False` → Only our new Dense layers are trained. Fast, safe, doesn't destroy ImageNet features.

Phase 2 (Fine-tuning): Unfreeze the top layers: `python base.trainable = True for layer in base.layers[:-20]: # Freeze all but last 20 layers layer.trainable = False model.compile(optimizer=tf.keras.optimizers.Adam(1e-5)) # Very small LR!` A very small learning rate (1e-5 instead of 1e-3) is crucial — otherwise the fine-tuned layers will overwrite learned ImageNet features.

Q37. What is the vanishing gradient problem? How does ReLU help?

Vanishing Gradient: In deep networks with Sigmoid or Tanh activations, gradients become very small (near 0) as they flow backwards through many layers. Early layers receive nearly no gradient → they can't learn → deep networks fail.

Why? Sigmoid's derivative maxes out at 0.25. Multiplying 0.25 × 0.25 × 0.25 across 10 layers = 0.000001 — effectively zero.

ReLU fix: For positive inputs, the gradient of ReLU is exactly 1.0. Multiplying 1×1×1×1 = 1 → gradient flows through without shrinking.

Dying ReLU problem: If inputs to ReLU are always negative, it outputs 0, has gradient 0, and that neuron never updates ("dies"). Fixed by: Leaky ReLU (small negative slope) or ELU.

Q38. Why do we use `n_mels=128` in the Mel-Spectrogram?

`n_mels=128` means we divide the frequency range into 128 Mel-scaled frequency bins.

More bins = more frequency resolution = more detail = larger array. The resulting spectrogram has shape `(128, time_frames)`.

We then resize this to `(224, 224, 3)` for MobileNetV2. 128 is a common choice — large enough to capture phonetic detail, small enough for efficient processing. Powers of 2 (64, 128, 256) are conventional in audio processing.

Q39. What is the learning rate? What happens if it's too high or too low?

The **learning rate** controls how much the model's weights are updated in response to the gradient.

`new_weight = old_weight - learning_rate × gradient`

Learning Rate	Effect
Too HIGH	Weights oscillate wildly, loss increases, training diverges
Too LOW	Training is very slow, may get stuck in local minima
Just right	Steady convergence to a good solution

In Gradient Boosting: Learning rate = shrinkage factor for each tree's contribution. Lower LR + more estimators = better generalisation but slower.

In fine-tuning (Transfer Learning): We use `1e-5` (very small) to avoid catastrophically overwriting pre-trained features (called "catastrophic forgetting").

Q40. What is an Epoch vs a Batch vs an Iteration?

Term	Definition
Epoch	One complete pass through the entire training dataset

Term	Definition
Batch	A small subset of training data processed together before updating weights
Iteration	One forward + backward pass for one batch

Relationship: `If dataset = 1000 samples, batch_size = 32: Iterations per epoch = ceil(1000 / 32) = 32 iterations If training for 20 epochs → 20 × 32 = 640 total iterations`

Why mini-batches? Processing the full dataset at once (batch gradient descent) requires too much memory. Processing one sample at a time (stochastic GD) is noisy. Mini-batches (32–256) balance memory and noise.

🔷 SECTION 7: Tricky / Advanced Viva Questions

Q41. What would happen if you didn't normalise images before feeding to MobileNetV2?

MobileNetV2's convolutional filters have weights learned assuming inputs in the [-1, 1] range. If you feed raw [0, 255] values: 1. Activations would be 255× larger than expected 2. ReLU would saturate 3. The model would output meaningless predictions Normalization is not optional — it must match exactly what was used during ImageNet pre-training.

Q42. Why do ensemble methods like Gradient Boosting outperform single decision trees?

A single deep decision tree has **high variance** — it overfits every noise pattern in training data. Different training samples would produce completely different trees.

> **Ensemble insight** (Bias-Variance decomposition): - Many weak learners (high bias, low variance) combined → reduce bias through boosting - Random perturbations (different data subsets, features) → reduce variance through averaging
>
> **Boosting specifically reduces bias** — each tree corrects systematic errors of the previous ensemble. Error approaches 0 with enough trees (as long as each tree is better than random).

Q43. What is One-Hot Encoding's "dummy variable trap"? How do you avoid it?

> If a feature has k categories, OHE creates k binary columns. These k columns are perfectly **multicollinear** — knowing the first k-1 tells you the k-th.
>
> For linear models (Logistic Regression), this causes numerical instability. Fix: Use `drop='first'` in OneHotEncoder to create only k-1 columns.
>
> **For tree-based models (Decision Tree, XGBoost, Gradient Boosting):** This doesn't cause problems — trees don't solve linear equations, so multicollinearity isn't an issue. We generally don't need to drop the first column.

Q44. What is the difference between `model.predict()` and `model.predict_classes()`?

> `predict_classes()` was a Keras convenience method that returned the class index directly (argmax).
>
> **It was removed in TensorFlow 2.6+.** The correct modern approach: `python probs = model.predict(x) # Returns probabilities [0.1, 0.8, 0.1] class_idx = np.argmax(probs, axis=1) # Returns [1] — index of highest prob`

Q45. If your model has 99% accuracy but is useless, what's wrong?

Class imbalance problem.

Example: 99% of loan applications are approved, 1% default. A model that **always predicts "Approved"** achieves 99% accuracy — but never catches a single defaulter.

Solution: - Use Recall, Precision, F1-Score, AUC-ROC instead of accuracy - Oversample minority class (SMOTE) - Undersample majority class - Use `class_weight='balanced'` in sklearn models - Adjust decision threshold

---

Q46. Why do we use `GlobalAveragePooling2D` instead of `Flatten` before Dense layers?

	Flatten	GlobalAveragePooling2D
What it does	Converts (7, 7, 1280) → vector of 7×7×1280 = 62,720 values	Averages each 7×7 map → vector of 1280 values
Parameters in next Dense	62,720 × 256 = 16M	1,280 × 256 = 330K
Overfitting risk	HIGH	LOW
Spatial info preserved	Yes	No

We use GAP because it dramatically reduces parameters and combats overfitting — especially important when our dataset is small relative to the model complexity.

Q47. What is `top_db=20` in `librosa.effects.trim()`?

Sounds below 20 dB relative to the peak amplitude are considered silence and removed from the beginning and end of the audio clip.

Why trim? Indian language audio clips might have leading/trailing silence from recording. This silence adds no language information but wastes processing. Trimming ensures the model focuses on actual speech content.

20 dB is a practical threshold — quiet background noise is typically under 20 dB below peak, while actual speech is above.

Q48. How does your Flask app handle cases where the model file is missing?

Lazy loading with error handling: ```python def get_model(): try: if models_dict['loan'] is None: models_dict['loan'] = pickle.load(open('pipeline.pkl', 'rb')) return models_dict['loan'] except FileNotFoundError: return None

@app.route('/api/loan/predict', methods=['POST']) def predict(): model = get_model() if model is None: return jsonify({'error': 'Model not loaded'}), 500 ... ``` Returning HTTP 500 with an error JSON lets the frontend show a user-friendly error message instead of crashing.

Q49. What is the difference between Keras `Sequential` and `Model` (Functional API)?

	Sequential API	Functional API
Structure	Linear stack of layers	Any DAG (directed acyclic graph)
Use case	Simple models	Multi-input, skip connections, shared layers

	Sequential API	Functional API
Usage	`model = Sequential([layer1, layer2])`	`x = layer(input); model = Model(inputs, x)`

We used Functional API for Transfer Learning: `python base = MobileNetV2(include_top=False, input_shape=(224,224,3)) x = GlobalAveragePooling2D()(base.output) x = Dense(256, activation='relu')(x) output = Dense(num_classes, activation='softmax')(x) model = Model(inputs=base.input, outputs=output)` This is necessary because MobileNetV2 is itself a complex non-sequential model.

Q50. What preprocessing do you do before feeding an image to the face mask model?

```python from PIL import Image import numpy as np

img = Image.open(uploaded_file) # Open file img = img.convert('RGB') # Ensure 3 channels (not RGBA/greyscale) img = img.resize((128, 128)) # Face mask model uses 128×128 arr = np.array(img) / 255.0 # Normalize to [0, 1] arr = np.expand_dims(arr, axis=0) # Shape: (1, 128, 128, 3) — batch dimension prediction = model.predict(arr) # Returns [[mask_prob, no_mask_prob, incorrect_prob]] ```

**Why `expand_dims`?** Keras models always expect a batch dimension. A single image has shape `(128, 128, 3)` but the model expects `(batch_size, 128, 128, 3)`. `expand_dims(arr, axis=0)` adds the batch dimension → `(1, 128, 128, 3)`.

---

📌 *Tip for the viva: Whenever asked "why did you choose X", always answer with: what X does + what alternatives exist + why X was better for your specific problem.*
```